Unit Testing — Brief Instructor Overview

What is Unit Testing?

Unit testing is a software testing practice in which the **smallest testable units of an application**—typically individual **functions, methods, or classes**—are tested **in isolation** to verify that they behave as expected.

A *unit* should have:

- A **single responsibility**
- **No dependency on external systems** (database, file system, network)
- **Deterministic output** for a given input

Why Unit Testing Is Important

Unit testing provides early feedback and long-term stability.

Key benefits

- Detects defects **early** in the development cycle
- Enables **safe refactoring**
- Improves **code quality and design**
- Acts as **living documentation**
- Increases **developer confidence**

What Is Tested in Unit Testing?

Typical targets include:

- Business logic methods
- Validation rules
- Calculation functions
- Service-layer logic (with dependencies mocked)

Not tested

- Databases

- External APIs
- File systems
- UI rendering
  (These belong to integration or end-to-end testing)

Unit Testing vs Other Testing Levels

Level	Scope	Dependencies
Unit Testing	Single method/class	Mocked
Integration Testing	Multiple components	Real
System / E2E Testing	Full application	Real

This aligns with the **testing pyramid**, where unit tests form the base and are the most numerous.

Core Principles of Good Unit Tests

- **Fast** – execute in milliseconds
- **Isolated** – no external dependencies
- **Repeatable** – same result every run
- **Readable** – easy to understand intent
- **Independent** – no shared state

Common Unit Testing Tools

- **.NET**: xUnit, NUnit, MSTest
- **JavaScript/TypeScript**: Jest, Mocha
- **Java**: JUnit, TestNG
- **Python**: PyTest, unittest

Example (Conceptual)

Scenario: Test a method that calculates discount

- Input: price = 1000, discount = 10%
- Expected Output: 900
- No database or API involved

If the output matches expectations, the unit test passes.